

Akai S900 / S950 Disk Format

Physical layout, directory, allocation table, sample files and program files — verified against 202 disk images, with the program parameters confirmed on an Akai S950 front panel.

1. Provenance

The disk, directory, allocation-table and sample-file layouts in sections 3–6 are documented at `mda.smartelectronicx.com/akai/akaiinfo.htm` ("Akai Disk & File Formats"). Every field listed here was independently verified by decoding real disk images: two Gotek/HxC sticks of 101 images each, 202 images in total, all decoding with zero CRC errors and zero unreadable sectors.

The **program file layout in section 7 is not covered by that document**. It was derived from the corpus and is stated here with the evidence supporting it.

Every count in this document is re-checked against the library by `docs/format-check.js` in the web repository, which fails if a figure no longer matches what the disks say. Last run: 20 September 2026, 38 claims, all holding.

2. Physical format

Property	Value
Container	HXCPICFE
HFE revision	0
Interface mode	12 (S950 double density)
Encoding	ISO/IBM MFM
Bit rate	250 kbit/s
Geometry	80 cylinders × 2 heads × 5 sectors
Sector size	1024 bytes (size code 3)
Sector IDs	1 to 5
Total capacity	819 200 bytes (800 KB)

Sectors are laid out linearly as:

$$\text{LBA} = (\text{cylinder} * 2 + \text{head}) * 5 + (\text{sector_id} - 1)$$

Within an HFE image, each track's MFM cell data for both sides is interleaved in 256-byte chunks — 256 bytes of side 0, then 256 bytes of side 1, repeating. Cells are stored least-significant-bit first. Sectors are framed conventionally: three `0xA1` sync bytes with a missing clock (cell pattern `0x4489`), then an address

mark — 0xFE for an ID field, 0xFB or 0xF8 for a data field — with CRC-16/CCITT (polynomial 0x1021, initial value 0xFFFF) computed across the three sync bytes, the mark and the payload.

2.1 The whole track, exactly

Reading a disk only needs the sectors found; *writing* one from nothing needs the entire track, gaps included. The layout below is not taken from the IBM System 34 spec but measured from the library, where all 101 disks agree without exception: a single identical 512-byte HFE header, 25000 bytes per track, sectors 1 to 5 in order with **no skew** on all 160 track/sides, and the first ID address mark at **bit 4928** of every one of them.

Each side holds 6250 MFM bytes:

```
gap 4a    0x4E x 230
sync      0x00 x 12
IAM       0xC2 0xC2 0xC2 0xFC      (sync cell pattern 0x5224)
gap 1     0x4E x 50

    five sectors, 1172 bytes each:
        sync    0x00 x 12
        ID      0xA1 0xA1 0xA1 0xFE, cyl, head, sector, size=3, CRC hi, CRC lo
        gap 2   0x4E x 22
        sync    0x00 x 12
        data    0xA1 0xA1 0xA1 0xFB, 1024 bytes, CRC hi, CRC lo
        gap 3   0x4E x 86

gap 4b    0x4E x 94

296 + 5 x 1172 + 94 = 6250
```

One detail matters only to a writer. A side is 12500 cell bytes, which is 48 whole 256-byte chunks and then 212 bytes of one, so **44 bytes of each side's last chunk are padding** that the interleave never addresses. The drive that wrote these disks left its gap fill running through them: the padding reads 0x49 0x2A repeating, which is exactly MFM-encoded 0x4E in this least-significant-bit-first cell packing. Filling those bytes with anything else — zeros, most obviously — still produces a perfectly readable image, but not an identical one.

That the layout is complete and correct is not argued but demonstrated: rebuilding each of the 101 disks from its 819,200 bytes of sectors alone, with no reference to the original file, reproduces that file **byte for byte** in all 101 cases.

3. Logical layout

The disk is addressed as 800 blocks of 1024 bytes. Block 0 carries both the directory and the allocation table; file data begins at block 4.

Offset	Size	Contents
0x000	1536	Directory — 64 entries of 24 bytes
0x600	1600	Allocation table — one 16-bit entry per block
0xC40	960	Unused
0x400	3072	Blocks 1–3, unused
0x1000	—	Block 4, first file data

A high-density disk extends the allocation table to 3200 bytes, occupying bytes 1536 to 4736.

4. Directory entry (24 bytes)

Offset	Size	Format	Description
0	10	ASCII	Filename. 0x00 in the first byte marks a free slot
10	6	—	Zero
16	1	ASCII	File type — see below
17	3	unsigned LE	File length in bytes, including the file's own header
20	2	unsigned LE	Starting block
22	2	—	S900 ID = {0, 0}

File types

Byte	Type	Notes
'S'	Sample	60-byte header followed by 12-bit sample data
'P'	Program	38-byte header followed by 70-byte keygroups
'D'	Drum set	Typically 262 bytes
'O'	Overall settings	Typically 40 bytes

Filenames are plain ASCII, space-padded — not the AKAI character set used by the S1000. Lower-case letters are legal on disk even though the front panel cannot enter them. Two files *of the same type* must not share a name, but a name may be reused across types, and in practice often is: naming a program after the single sample it plays is a common convention on these disks. DSKA0044 does it thirteen times — GLOCK 1, MARIMBA 2 and eleven others each exist as both a program and a sample — and DSKA0057 and DSKA0080 do the same. The reference document's flat "must not share a name" is too strong; a writer that enforces it across the whole directory will refuse to reproduce the library's own naming.

The keygroup and sample arena

Keygroup records and sample descriptors share one region of the sampler's RAM, and the zone pointers of section 7 are addresses within it. The arrangement is entirely determined by the directory: each

program's keygroups are packed 70 bytes apart in directory order, a *single empty 70-byte record* separates consecutive programs, and the sample descriptor table follows the last program. So

```
keygroup k of program j = arena + 70 x (sum over i<j of (keygroups_i + 1) + k)
sample descriptor table = arena + 70 x (keygroups + programs - 1)
```

The empty record between programs is what makes the $- 1$ correct: there is one separator after each program except the last. Checked against the library, this predicts all 1774 keygroup chain pointers on all 98 disks that have them, and the table base on every disk, exactly.

The arena start is not stored anywhere. It is 0xC5F6 on 97 of the 98 disks that reveal it; DSKA0023 uses 0xC47E and the two sample-free KEYGRP PGM disks use 0xAB90, so it must be read back from the disk rather than assumed — from the zone pointers where there are any, since those are the ones the sampler follows, and from the keygroup chains otherwise.

A writer must recompute these pointers, not adjust them. Both of this project's implementations originally shifted the stored values by a delta whenever a program changed size. That conflated the chain pointers with the zone pointers, which live in the same arena but are reached differently, and the error compounded across edits: one disk ended up with its chains 27,600 bytes below the arena and two programs sharing a record, which crashed the S950 on load while every other structure on the disk checked out. Deriving the layout from the directory cannot drift.

The directory has no holes

Entries occupy slots $0..n-1$ with nothing empty in between. All 100 images in the library are like this, and they are grouped by type in the order programs, overall settings, drum set, samples — 98 disks are exactly that sequence and the remaining two hold programs only. Nothing in the format *requires* contiguity, but the sampler appears to rely on it: a disk that had four samples deleted, leaving slots 4–7 blank while ten files sat above them, crashed the S950 on load even though every other structure on it checked out — its allocation table, chains, headers, RAM addresses and zone pointers were all correct. Closing the gap, and changing nothing else, made the same disk load. The likely reading is that the loader walks the directory until the first empty slot and stops, so the programs it had already read went on to reference samples that were never loaded.

The practical rule for a writer: deleting a file must shift every later entry down a slot rather than simply blanking the one being removed. Because the relative order is preserved, the position-based zone pointers of section 7 stay correct across the move.

5. Allocation table

One unsigned 16-bit little-endian entry per block, starting at byte 1536. The entry for a block holds the number of the next block in that file's chain.

Value	Meaning
0x0000	Block is free
0x8000	End of chain — this is the file's last block
other	Next block in the chain

To read a file: start at the directory entry's starting block, follow the chain until 0x8000, concatenate the blocks, then truncate to the directory's length field. Chains are usually contiguous but must not be assumed to be. A reader should guard against loops and out-of-range links.

6. Sample files

6.1 Header (60 bytes)

Offset	Size	Format	Description
0x00	10	ASCII	Filename
0x0A	6	—	Name padding. Zero on all but 13 samples, which pad with ASCII space instead — the same two conventions the program header uses
0x10	4	unsigned LE	Number of sample words (N)
0x14	2	unsigned LE	Sample rate in Hz
0x16	2	unsigned LE	Tuning, in 16ths of a semitone. The panel splits it: nominal pitch is the value divided by 16, a MIDI note number (C3 = 60, stored 960), and fine pitch is the remainder, 0..15
0x18	2	signed LE	panel . Sample loudness. <i>Not</i> zero as the reference document has it — non-zero in 200 of 1110 samples, spanning -31..+50
0x1A	1	ASCII	Loop mode — 0 one-shot, L loop, A alternating
0x1B	1	—	Zero
0x1C	4	unsigned LE	End marker
0x20	4	unsigned LE	Start marker
0x24	4	unsigned LE	Loop length
0x28	2	unsigned LE	Pointer to the sample's loop descriptors in RAM. Base 0xB6F4, 10 bytes per descriptor. See below
0x2A	1	—	Zero
0x2B	1	ASCII	Loop direction — N normal, R reverse, in the same style as the loop mode at 0x1A. Only DSKA0083 PHONE 3 is reversed, which is why the byte looks constant
0x2C	10	—	Zero
0x36	3	unsigned LE	Address of the sample in sample RAM. See below
0x39	3	—	Zero

The two bytes at 0x18 are listed as zero by the reference document. They are not: 200 of the 1110 samples carry a value there, a signed 16-bit little-endian quantity spanning -31 to +50. It is the **sample loudness**,

confirmed on the panel — DSKA0002 TONE 1 reads loudness 30 and stores 30.

The same panel page resolves the tuning word at 0x16. It is displayed as two figures rather than one: nominal pitch, the value divided by 16 and shown as a note name, and fine pitch, the remainder. TONE 1's 960 is nominal pitch 60 (C3) with fine pitch 0. Across the corpus the nominal part spans MIDI 24 to 96 with no value outside the valid range, and the fine part is non-zero on 71 samples.

How the three loop fields combine

The header carries an end marker, a start marker and a loop length, and the names invite the wrong reading. The machine loops **backwards from the end marker by the loop length** — it plays end - length . . . end round and round — and the start marker at 0x20 is *not* where the loop begins. It is the point playback starts from, and it is simply 0 in **250 of the 324** looped samples; it equals end - length in only **8**. A player that treats the start marker as the loop start gets 316 of 324 wrong.

Nor does a loop have to run to the end of the sample. The end marker equals the sample length in **208 of 324**, so a third of them stop short — and **30** stop short by more than 50 ms, with a median tail of **277 ms**. That tail is the release: what sounds after the key comes up and the loop is left. Looping to the end of the sample instead plays the release round and round, and leaves none of it to hear.

DSKA0051 BOWEDBASS is the clearest case: all nine of its samples loop before the end, leaving between 0.12 and 0.48 s of tail. Its TONE E1 is 53,496 words long, ends its loop at 43,437 with a length of 10,214 — so it loops 33,223 .. 43,437 and lets the last 10,059 words play out once.

Where the length would reach back past the start of the data it would be nonsense; no sample in the library does, so nothing here says what the machine would do about it.

The pointer at 0x28 indexes a table of **10-byte loop descriptors** in RAM. Every disk's first sample points at 0xB6F4, and the gap to the next sample's pointer is always a whole number of 10-byte records — between 2 and 9 of them. How many a sample consumes follows from its loop mode and its size. Writing *f* for the number of whole 128 KB pages the sample data fills (*f* = floor(2 × words / 131072)):

Loop mode	Records consumed	Fit
One-shot 0	2 + <i>f</i>	723 of 725
Looping L	3 + <i>f</i>	267 of 270
Alternating A	3 × (1 + <i>f</i>)	12 of 16

1002 of the 1011 consecutive spacings in the corpus, and the shape of it makes sense: a one-shot needs a descriptor pair, a loop needs a third for the loop itself, and an alternating loop needs the whole set repeated for every page the sample spans — which is what bidirectional playback would require, since direction has to be tracked within each page. The nine exceptions are all samples above 128 KB.

Splitting the corpus at that threshold sharpens the picture. Below 128 KB of RAM the count is not merely a good fit but exact, and simpler than the formula above: **2 records for a one-shot, 3 for a loop**, alternating included — 651, 236 and 6 consecutive pairs respectively, with no exception. At or above 128 KB it is not a function of the header at all: two one-shots of 280,000 bytes each, CHINA on DSKA0016 and TR808 CYM on DSKA0007, consume 3 and 4 records, and padding out the rule with the start address or the number of page boundaries crossed does not separate them either. The likeliest explanation is that the descriptor count reflects how many loop points the sample actually carried — the S950 allows several

per sample — which is held in the sampler rather than in these 60 bytes. A writer should treat the formula as exact below 128 KB and as an estimate above it.

That estimate turns out not to matter. A disk written with every *keygroup* chain pointer — the equivalent RAM pointer in the program records — deliberately corrupted to 0x5A5A still plays its programs correctly on an S950. The sampler rebuilds its RAM arena when it loads a disk rather than reading the stored addresses, so these fields record where things sat when the disk was saved; they are not instructions to be obeyed. Write them to match what the hardware would have written, but an approximation cannot break a disk. Only the chain pointers were corrupted in that test, the sample address and descriptor pointer being the same class of field rather than separately proven.

The three bytes at 0x36 give the address the sample occupies in the sampler's sample memory, and the arithmetic is exact. On 99 of the 101 disks the first sample sits at 0x18000, and taking a disk's samples in directory order each subsequent address follows from the one before by

```
ceil(2 × words / 16) × 16
```

in **1011 of 1011** consecutive pairs across the corpus. Two consequences follow. Samples are stored back to back in memory in disk order, on a 16-byte grid. And a sample occupies *two bytes per word* in RAM against one and a half on disk: the 12-bit packing of 6.2 is a disk format only, expanded to 16-bit words when loaded.

The directory length relates to the sample count exactly:

```
directory_length == 60 + 1.5 * N
```

This holds for all 1110 samples on the corpus without exception, which both confirms the sample-count field and establishes that data is stored at 12 bits per sample.

6.2 Sample data — 12-bit signed, split packing

The payload is $1.5 \times N$ bytes. The packing is unusual: the sample is split in half, and the low nibbles of *both* halves are interleaved through the first N bytes, with the second half's high bytes following at the end.

First N bytes, read as $N/2$ pairs. For pair i ($i = 0 \dots N/2-1$):

```
byte[2i]    high nibble = low 4 bits of word i
byte[2i]    low  nibble = low 4 bits of word (N/2 + i)
byte[2i+1]  = high 8 bits of word i
```

Then $N/2$ bytes:

```
byte[N + i] = high 8 bits of word (N/2 + i)
```

Reassemble, and sign-extend from 12 bits:

```
word = (high8 << 4) | low4
value = (word >= 2048) ? word - 4096 : word
```

Verification — decoding one sample three ways and measuring the mean absolute difference between consecutive values (a correct decode yields a smooth waveform; a wrong one yields noise):

Interpretation	Peak amplitude	Mean sample delta
16-bit little-endian words	18983 — outside 12-bit range	1452
Naive 12-bit pairs, 2 per 3 bytes	2048	1028
Split packing, as above	1398 — within range	112.8

7. Program files

Not covered by the reference document. Everything in this section was derived from a corpus of 390 programs and 1908 keygroups, then checked field by field against an Akai S950 front panel; fields marked **panel** were read directly off the hardware. Figures given as "of 1899" exclude DSKA0049, the one image whose contents differ between the two sticks; those given as "of 1908" cover every keygroup on both.

A program is a fixed header followed by one 70-byte record per keygroup:

```
file_length == 38 + 70 * keygroup_count
```

All 391 distinct programs across both sticks satisfy this exactly, in uniform steps of 70, from 108 bytes (1 keygroup) to 4518 bytes (64 keygroups). The program header begins with the program name in the first 10 bytes, matching the directory entry.

7.1 Program header (38 bytes)

The header is the first 38 bytes of the file. Six of its fields are identifiable from the corpus alone, and two of them are exact.

Offset	Field	Notes
0 .. 9	Program name	10 chars ASCII, matching the directory entry
10 .. 15	Name padding	ASCII space in all 341 programs an S900 wrote, 0 in all 49 an S950 wrote
16, 17	KEY → loudness	panel , signed 16-bit. Zero in 386 of 390 programs. DSKA0078 SOFT FLUTE reads −6 and stores 250, 255 — the sign extension a 16-bit field needs. See the note below on the one exception
18, 19	Load address	Unsigned 16-bit, 0xAB90..0xD3A2. See below
20	Reserved	Constant 0
21	Positional crossfade	panel . 0 off, 255 on. Of the multi-keygroup programs that set it, 92% have overlapping keygroups against 32% of those that do not
22	Format marker	255 in all 341 S900 programs, 0 in all 49 S950 ones — the same split as the filter envelope in 7.4, with no exceptions
23	Keygroup count	Equals the count implied by the file length in 390 of 390 programs
24, 25	Reserved	Constant 0
26	MIDI program number	panel , zero-based here and shown counting from 1: DSKA0002 SNARE 1 stores 1 and reads MIDI Prog 2. Matches the program's position on its disk in 371 of 390; where it does not it preserves creation order, as on DSKA0022 where the five KICK-SN programs carry 4, 0, 1, 2, 3 while sitting in the directory as P5, P1, P2, P3, P4
27	unknown	255 in 388 of 390
28 .. 37	Reserved	Constant 0

Only four programs in 390 set KEY → loudness at all, and three of them read cleanly as a signed 16-bit value: +10, −6 and −5, with byte 17 holding 0xFF on the two negatives. The fourth, DSKA0061 THE ISLAND, stores 254 with byte 17 at 0, which reads as +254 — an order of magnitude outside the range of anything else in the corpus, and outside what the panel offers. A stale byte in an otherwise unused field is the likeliest account. It is worth flagging rather than smoothing over, because a writer that sets only the low byte would produce exactly this.

The load address is the most informative field, because its *increments* are exact. Taking the programs on a disk in directory order, the address of each advances by

$$70 \times (1 + \text{keygroup_count})$$

in **289 of 289** consecutive pairs across both sticks. Two things follow. The value is a RAM address, not a disk offset — programs land nose to tail in the sampler's memory in the order the disk holds them, and the first one on a disk usually lands at 0xC5F6. And the header occupies **70 bytes in memory** though only

38 on disk: the same size as a keygroup. The 38-byte on-disk header is a truncation of a 70-byte record, which is consistent with its last ten bytes being constant zero.

A writer must maintain both the load address and the keygroup count. They restate what the rest of the file already implies, and the sampler believes the header rather than recounting. Both are exact on all 100 library disks. Getting them wrong is not a cosmetic error:

- A **stale count** (byte 23) silently truncates the program. A disk whose first program had grown from four keygroups to eight still said four, and the sampler played only the first four — the samples in the rest of the program were simply not there.
- A **stale load address** (bytes 18..19) overlaps programs in RAM. On the same disk the second program still loaded at 0xC754, the address that was correct before the first program grew by four keygroups, so its records landed on top of the first program's. That crashed the S950 on load.

Both follow from the arena layout of section 5: a program's load address is $\text{arena} + 70 \times (\text{its first record index})$, and its count is simply how many keygroup records the file holds.

7.2 Keygroup record anatomy

The 70 bytes divide into a header, two velocity zones of equal size, and a link:

Offset	Size	Contents
0 .. 23	24	Keygroup header — key range and the amplitude envelope
24 .. 45	22	Velocity zone 1 — sample name and its per-sample parameters
46 .. 67	22	Velocity zone 2 — same layout
68 .. 69	2	Link to the next keygroup

An unused second zone carries the placeholder name 2 SAMPLE, which occurs in 87% of keygroups.

7.3 Keygroup header (24 bytes)

Fields marked **panel** were read directly off an S950 front panel and match the stored bytes exactly.

Offset	Field	Notes
0 / 1	High key / low key	MIDI note numbers, C3 = 60
2	Velocity switch threshold	panel. The velocity at which zone 2 takes over from zone 1. The panel displays the number itself, and 128 is simply its top — unreachable, since MIDI velocity stops at 127. See 7.5
3 . . 6	VCA attack / decay / sustain / release	panel , 0..99
7	VEL SENS → filter	panel
8	KEY → filter	panel. Default 50; reads 0 as "Off"
9	VEL SENS → attack	panel
10	VEL SENS → release	panel. Reads –50 on DSKA0006 DRUM–1 keygroup 4, which stores 206. Signed, and spans exactly –50..+50 across the corpus; zero in all but 20 of 1899 keygroups, which is why it took a deliberately chosen program to see
11	VEL SENS → loudness	panel. The byte that shifts across whole drum kits between the factory DRUM-A/B/C/D variants
12	WARP velocity	panel , default 0 (warp off)
13	WARP attack offset	panel , signed
14	WARP time	panel
15	LFO delay	panel
16	LFO rate	panel
17	LFO depth	panel
18	Flags	panel. Bit 0 constant pitch (481 keygroups), bit 2 LFO desync (1652), bit 3 one-shot playback (163). Bit 1 is set in one keygroup of 1908 and is unidentified; bits 4 to 7 are never set
19	Output assignment	panel. Panel value is this byte plus one, 0xFF wrapping to 0: 0 all, 1..8 the individual mono outputs, 9 left, 10 right. See 7.6
20	Reserved	Zero but for one keygroup of 1908
21	LFO depth → aftertouch	panel. Constant 0 across all 1908 keygroups — no library program uses aftertouch — so the corpus corroborates but cannot locate it
22	LFO depth → modwheel	panel , default 50; 427 of 1908 keygroups change it
23	VCF envelope amount	panel. Signed, and spans exactly –50..+50 across all 1908 records with no outliers

7.4 Filter envelope (bytes 34 .. 37)

A keygroup-level setting, though it is stored inside zone 1's span:

Offset	Field	Notes
34 .. 37	VCF attack / decay / sustain / release	panel , 0..99. Blank (ASCII space) on 87% of disks: the S900 had no filter envelope, so only S950-era programs write these bytes

The matching bytes in zone 2 (56 – 59) are **blank in all 168 keygroups that use a real second zone** — never written, never differing — which is what establishes this as a keygroup parameter rather than a per-zone one.

The front panel lays this page out as attack, decay, sustain, release and then AMNT, which invites the guess that the amount is stored in byte 38, immediately after the envelope. It is not. Across the 252 keygroups that write a filter envelope, bytes 38 and 39 are **always zero** — the only other value either takes is ASCII space, in the five records that are only partly written — whereas byte 23 carries ten distinct signed values spanning –50 to +50, and varies exactly where one would expect it to: DSKA0003 MIX HATS pairs an envelope of 0/28/30/60 with an amount of 44, while KICK–SN MI writes 31/99/99/60 with an amount of 0. The panel's grouping and the file's layout are simply not the same thing: the envelope sits inside zone 1's span and the amount stays in the keygroup header.

Worth noting too that 166 of those 252 write an envelope but leave the amount at zero, so the sampler evidently writes these bytes whenever an S950 saves the program, whether or not the envelope does anything.

7.5 Velocity zone (22 bytes)

Offsets relative to the start of the zone: add 24 for zone 1, 46 for zone 2. Unlike the filter envelope, these fields *are* genuinely per zone — across the 168 keygroups with a live second zone, the filter differs from zone 1 in 91 and loudness in 43.

Offset	Field	Notes
+0 .. +9	Sample name	10 chars ASCII
+10 .. +13	—	Zone 1 only: the filter envelope, see 7.4
+14, +15	Reserved	Never written — see 7.8
+16, +17	Internal sample reference	Unsigned 16-bit. A given sample name always carries the same value within a disk — 1862 pairs over 1065 names, zero conflicts. Not a user parameter
+18	Fine tune	panel . Unsigned 0..255, a fraction of a semitone <i>upward</i>
+19	Transpose	Signed whole semitones, –4..+3 across the corpus
+20	Filter cutoff	panel , 0..99, default 99 (fully open)
+21	Loudness	panel , signed, default 0

The threshold that selects between the two zones is not in the zone record at all: it is **byte 2 of the keygroup header**. DSKA0082 CAR HORN stores 115 there and the panel reads a velocity switch of 115.

The corpus confirms it without ambiguity. Of the 266 keygroups that name a second sample, 45 carry a threshold in the range 60–116. Of the 1633 that name no second sample, **not one** carries anything but 128. That is not a sentinel: the panel shows 128 as a number like any other. It is simply the top of the range, and one above the highest MIDI velocity, so a keygroup left there can never reach zone 2. The remaining 221 two-sample keygroups also sit at 128, leaving zone 2 silent; on DSKA0082 that is plainly a template left over from copying, since four of its programs carry an unheard CUCK00 in zone 2 at the same loudness.

Tuning is split across the pair at +18 and +19, and only one of them is signed. Transpose (+19) holds signed whole semitones. Fine (+18) is **unsigned**: the panel displays 128 for the byte 128, not –128. The pitch offset is therefore `transpose + fine/256` semitones, and a fine value can only move a note *up*.

The corpus shows the consequence plainly. A small downward detune has to be written as a whole semitone down plus a large fraction back up, so at transpose –1 the fine value is non-zero in 206 of 211 keygroups and averages 163, whereas at transpose 0 it is zero in 1396 of 1574 and averages 74 where set. Of the 493 non-zero fine values, 340 are multiples of 16 — a sixteenth of a semitone, or about 6 cents, appears to be the step most players used, though the panel accepts finer: DSKA0078 RECORDER 1 reads 2.

7.6 Byte 19: the output assignment

The panel setting is this byte plus one, with 0xFF wrapping round to 0:

Stored	Panel	Meaning
0xFF	0	All — the default, 1608 of 1899 keygroups
0 .. 7	1 .. 8	Individual mono outputs
8	9	Left
9	10	Right

Eleven panel settings, eleven stored values, every one of them present in the corpus. The rest of this section is the argument that led to the same answer from the disk images alone, before the panel confirmed it.

Byte 19 holds 0xFF in 1608 of the 1899 keygroups. Where it does not, it takes only the ten values 0 to 9 — and the way those values are distributed identifies the field without any need for the front panel.

Drum kits give each keygroup its own value: TOM CYM 2 on DSKA0009 runs 3, 2, 1, 0 across its four toms and 4, 5, 6 across its three cymbals, and HIPHOP PRC on DSKA0007 runs 0 to 6 across its seven percussion hits. That is the signature of a per-keygroup routing assignment, not of a sound parameter.

The decisive case is DSKA0042. It carries TUBULARBEL, four keygroups covering notes 24–127 with byte 19 unset, and TUBULAR 2, which is the same four keygroups *twice* over identical key ranges and with identical envelopes, filter and loudness. The two copies differ in exactly two respects: the second is fine-tuned +2, and its byte 19 reads 9 where the first reads 8. A detuned unison double is only worth making if the two halves are separated in the output — so 8 and 9 are a left/right pair. PIZ-CHORUS on DSKA0064 agrees: a pizzicato across the full keyboard on 9, layered with a chorus sample on 8.

With 8 and 9 spoken for as left and right, 0 to 7 line up with the eight individual outputs, and 0xFF is the unassigned default. Consistently with that reading, the two ranges do mix within a kit — LINN 9000 on DSKA0023 puts four of its drums on numbered outputs and the remaining twenty-four on 9.

7.7 Corpus-wide corroboration

Fields established from the images alone, before any panel check:

Offset	Field	Evidence
0	High key	Byte 1 $\leq$ byte 0 in all 1908 records, no exceptions. Ranges tile consecutively across the keygroups of multi-sampled programs, and each sample's own tuning value always falls inside its keygroup's range
1	Low key	
24 . . 33	Zone 1 sample name	1744 of 1908 match a sample in the same disk's directory; none blank or malformed, the rest are valid names on other disks
46 . . 55	Zone 2 sample name	Same 10-byte ASCII form, exactly 22 bytes after zone 1
40 . . 41	Zone 1 sample reference	Unsigned 16-bit. A given sample name always carries the same value within a disk — 1862 name/value pairs over 1065 distinct names, zero conflicts . The same sample seen in another keygroup's zone 2 carries the identical value. An internal address, not a user parameter
62 . . 63	Zone 2 sample reference	
68 . . 69	Next keygroup	Unsigned 16-bit, and 0 in the final keygroup of all 390 programs without exception. Consecutive keygroups usually differ by exactly 70

7.8 What remains unknown

One byte of the seventy is unaccounted for: byte 20, which is 0 in all but one of the 1908 keygroups and is treated as *reserved*. Byte 21, constant 0 throughout, turned out not to be reserved at all — it is the LFO depth from aftertouch, which no program in this library ever uses. That is a caution worth recording: a byte being constant across 202 disks is evidence that nobody changed it, not evidence that it means nothing.

The single keygroup carrying byte 20 = 1 is DSKA0013 CSDS 1510 keygroup 4. It is also the one keygroup in the corpus that names a second sample, sets a velocity threshold for it, and yet leaves the sample reference at zero — but 101 other keygroups name an unresolved second sample without setting byte 20, so that is coincidence rather than explanation. The likeliest reading is a stray bit rather than a parameter.

What is genuinely still open is not a byte but two **bits**, described next.

Byte 18 is a flag byte, resolved from the keygroup page that follows LFO desync on the panel. Bit 0 is **constant pitch**, bit 2 **LFO desync**, bit 3 **one-shot playback**. Only bit 1 is left, set in a single keygroup of 1908; bits 4 to 7 are never set.

Both new bits read correctly against the corpus. DSKA0009 TOM CYM 2 holds 4 across its four toms and 5 across its three cymbals — constant pitch off for the pitched toms and on for the cymbals. One-shot playback appears in only eleven programs, and every one of them is a drum kit or a sound-effects bank: ROCK KIT 1, DRUMKIT 1 and 2, TELEPHONE, UK HUMOUR, BLOOD+GUTS among them.

This also corrects an earlier inference. Bit 0 had looked like a play-to-end flag because 336 of the 350 keygroups setting it reference a one-shot *sample*. That correlation is real but incidental: constant pitch is what you use for percussion, and percussion samples do not loop. The flag that actually governs playing to the end is bit 3.

Eight further bytes are accounted for as *reserved* rather than unknown: zone 1's +14 and +15, and the whole of zone 2's +10 to +15. All eight hold 0 in every one of the 238 keygroups an S950 wrote and ASCII space in all 1661 an S900 wrote, with no exceptions in either direction. They track the same six-byte block the filter envelope occupies, and were evidently allotted but never given a meaning.

7.9 Relationship to the S1000

The S1000 program format — a 150-byte header followed by 150-byte keygroups — is documented on the same site but **does not apply**: it fits none of the 391 program files measured here, and the S950 reverses the key-range order, putting high key before low key. Its parameter list nonetheless remains the best available guide to the undecoded bytes, since the machines are directly related. Note that the S1000 carries four velocity zones per keygroup where the S950 carries two.

8. Drum set files — layout only

Every disk carries one file of type D, named DRUM SET, and every one is exactly 262 bytes. The layout follows from that length and is confirmed on all 100 drum sets in the corpus:

Offset	Size	Description
0x00	22	Header. Byte 0 is 0xFF when the drum set is in use and 0 otherwise; the other 21 bytes are zero on every disk
0x16	8 × 30	Eight voice records

$22 + 8 \times 30 = 262$ exactly. Within a voice record:

Offset	Size	Format	Description
0	2	unsigned LE	Voice number, 0..7 in order
2	1	unsigned	MIDI note this voice answers to
3	23	—	Voice parameters, undecoded
26	2	unsigned LE	RAM pointer, base 0x1040, stepping 512 per voice
28	2	unsigned LE	RAM pointer, base 0x3040, stepping 256 per voice

The parameters are undecoded because there is almost nothing to decode them from. Across 800 voice records in the library there are only **ten** distinct parameter blocks: 672 records carry one default block, 96 carry a mostly-zero variant, and the remaining 32 are eight blocks belonging to a single drum kit duplicated across DSKA0004, 0005, 0017 and 0018 — the only four disks whose header byte marks the drum set as in use. Nobody who made these disks used the drum mode.

That those four are a factory default rather than four real kits can be shown, not just assumed: the disks carry **5, 12, 22 and 26 samples** respectively, and no sample name appears anywhere in the 262 bytes, yet all four kits are byte-for-byte the same. A kit that actually addressed a disk's own samples could not be identical on a disk with five of them and one with twenty-six. So the library contains no curated drum kit at all — the useful count of examples to decode from is zero, not one.

What that one kit does establish is the note field. Its eight voices carry notes 48, 52, 55, 57, 59, 69, 71 and 74 — C2, E2, G2, A2, B2, A3, B3, D4 — ascending, which is a kit laid out across the keyboard and cannot reasonably be anything else. Everything from offset 3 to 25 is constant within that kit apart from a handful of bytes, and with a single example there is no way to separate meaning from coincidence. Decoding the rest needs an S950 front panel in drum mode, the same method that resolved the ambiguous keygroup bytes in section 7.

The first two bytes of each voice are **not** a reference to anything: across all 99 disks that carry a drum set they read 0, 1, 2 ... 7 without exception, so they are just the voice ordinal. That matters to a writer, because it means **nothing on a disk points at a program** — not the drum set, not the overall settings, and no address or count field anywhere outside a program's own header. Deleting a program therefore needs only to free its blocks, close the gap in the directory and rebuild the arena around one fewer program.

9. Cross-checks

Three fields validate against independent evidence, which is the strongest confirmation available short of hardware testing.

Field	Evidence
Tuning	On one disk the samples CHOP G1, CHOP C2, CHOP F2 and CHOP A2 carry 43, 48, 53 and 57 — exactly the semitone intervals their names imply, with C3 = 60
Loop mode	Across 1110 samples only 0, L and A appear (786 / 306 / 18). No other value occurs
Sample rate	The 1104 samples in the library use 42 distinct rates spanning 11,773 to 44,329 Hz — not the handful of round settings an earlier reading of this field suggested. Common values are round (20000 x25, 22500 x15, 17500 x7) but plenty are not (11773, 22170, 23750), so a reader must treat the field as a free 16-bit value rather than an index into a table of presets. The library's range is not the machine's: the S950 itself will run down to 7500 Hz , so a writer should take that as the floor, not the lowest rate a disk happens to contain

10. Reading a disk — summary

1. Decode the HFE track data to 800 sectors of 1024 bytes.
2. Order them: $LBA = (cyl * 2 + head) * 5 + (sector_id - 1)$.
3. Read 64 directory entries from offset 0x000.
4. For each entry with a non-zero first byte and a known type:
 - a. follow the allocation chain from the starting block until 0x8000;
 - b. concatenate the blocks and truncate to the length field;
 - c. parse the file header according to its type.